# Dev2 전체 실행 플로우

날짜: 2026-05-29 복습 예정: D+1, D+3, D+7

---

## 1단계 — 환경 준비 (서버 시작 전)

```
Windows 필수
  └── docker context use default
        (desktop-linux context → default로 전환, 안 하면 Docker 연결 오류)
```

---

## 2단계 — DB 시작

```
docker compose up -d postgres
        │
        ▼
pgvector/pgvector:pg16 컨테이너 기동
        │
        ▼
volumes: ./migrations:/docker-entrypoint-initdb.d
        │
        ▼  (postgres 최초 시작 시 자동 실행)
SQL 파일 자동 실행 → 테이블 생성
  • scan_executions  (스캔 실행 이력)
  • scan_issues      (개별 이슈)
  • scan_summaries   (심각도별 집계)
  • chunk_embeddings (pgVector RAG용)
```

---

## 3단계 — FastAPI 서버 시작

```
python -m uvicorn dev2.main:app --port 8005
        │
        ▼
main.py — 라우터 3개 등록
  • scan_router    → POST /scan, POST /scan/mock-bandit
  • insights_router→ GET  /insights/developer/{id}, /insights/team, /insights/
trends
  • issues_router  → PATCH /issues/{id}/dismiss
  • /health        → GET  {"status":"ok","service":"dev2"}
```

---

## 4단계 — 요청 수신 (두 가지 경로)

### 경로 A — 정식 스캔 (Prototype 1)

```
Dev1 Orchestrator
  └── POST /scan + TaskPayload JSON
            │
            ▼
```

```
      FastAPI 파싱 & Pydantic 검증
            │
            ▼
      _payload_to_state() — TaskPayload → Dev2State 변환
            │
            ▼
      BackgroundTasks.add_task(run_dev2, state)
            │
            ▼
      202 Accepted 즉시 반환 ← 클라이언트가 기다리지 않아도 됨
            │
            ▼  (응답 전송 후 백그라운드에서 실행)
      run_dev2(state) → LangGraph 5노드 파이프라인 시작
```

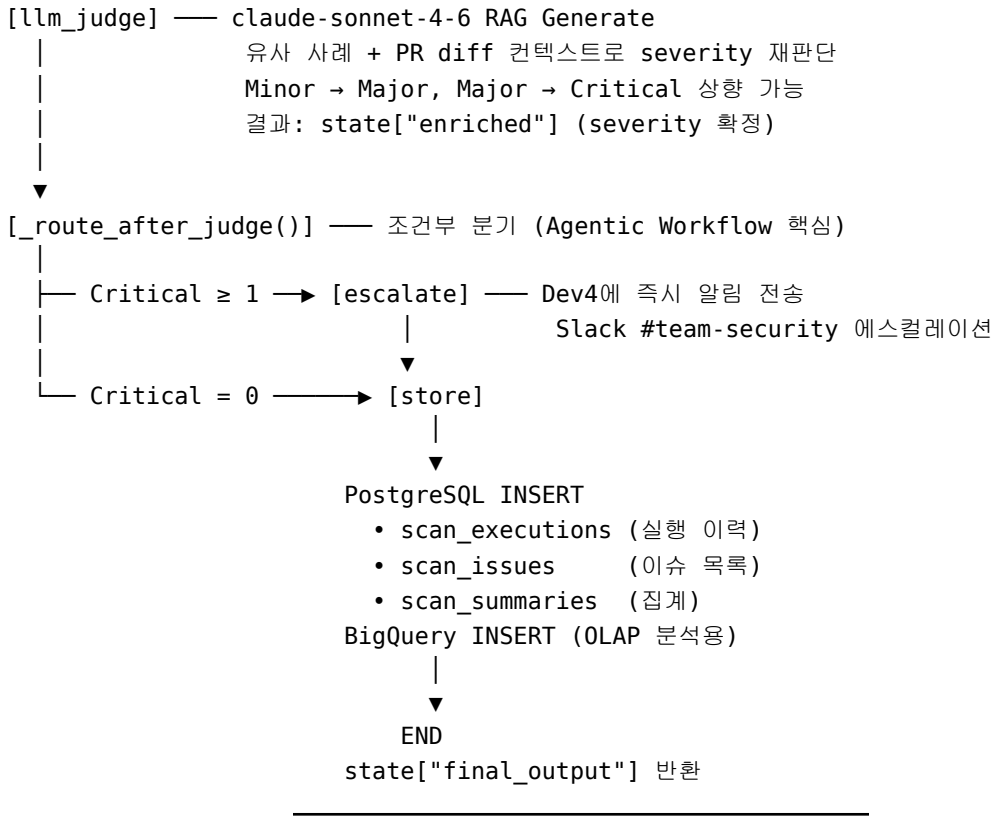
### 경로 B — Mock Bandit 스캔 (Prototype 2)

```
curl -X POST /scan/mock-bandit
  (TaskPayload 없이 호출 가능 — MVP 테스트용)
        │
        ▼
  asyncio.to_thread(subprocess.run, ["bandit", ...])
        │
        ▼
  bandit_result.json 생성
        │
        ▼
  JSON 파싱 → issues 리스트 + summary 딕셔너리
        │
        ▼
  db.py 함수 직접 호출 (경로 A와 달리 LangGraph 안 거침)
        │
        ▼
  200 OK + {"status":"success","scan_id":N,"issue_count":10,...}
```

---

## 5단계 — LangGraph 5노드 파이프라인 (경로 A만)

```
START
  │
  ▼
[run_tools] ─── ESLint / Pylint / Bandit / pip-audit 병렬 실행
  │               PR diff에서 +줄만 추출 → 각 도구로 분석
  │               결과: state["static_raw"], state["security_raw"]
  │
  ▼
[vector_search] ─── pgVector RAG Retrieve
  │                   이슈와 유사한 과거 사례 검색 (chunk_embeddings)
  │                   결과: state["enriched"] (유사 이슈 첨부)
  │
  ▼
```




## 6단계 — 결과 조회

```
GET /insights/developer/{developer_id}
        │
        ▼
  scan_executions + scan_issues JOIN
        │
        ▼
  {"developer_id":..., "total_issues":..., "critical":..., ...}

GET /insights/team
        │
        ▼
  전체 팀 집계 통계

PATCH /issues/{id}/dismiss
        │
        ▼
  오탐(false positive) 마킹 → dismissed=true
```

## 전체 요약 (한눈에)

```
[환경준비] docker context use default
    ↓
[DB]    docker compose up -d postgres  →  테이블 자동 생성
    ↓
[서버]  uvicorn dev2.main:app --port 8005
    ↓
[요청]  POST /scan (Dev1) 또는 POST /scan/mock-bandit (테스트)
    ↓
[분석]  run_tools → vector_search → llm_judge → (escalate?) → store
    ↓
[저장]  PostgreSQL scan_executions / scan_issues / scan_summaries
    ↓
[조회]  GET /insights/* → 통계 반환
```